

Photobox CameraBridge – Worker (worker.exe) – Handbuch

Stand: 2026-03-25

Zusammenfassung

- zentrale Kameraschnittstelle des Systems - kapselt die eigentliche Kamerasteuerung - stellt Funktionen per Named Pipe IPC bereit - wird vom API-Server über IPC angesprochen

1. Überblick

Der Worker übernimmt im Kern:

- Kameras suchen (**refresh**)
- Kamera auswählen (**select**)
- LiveView starten und stoppen
- LiveView-Frames für den API-Server bereitstellen
- Kamera-Settings lesen und schreiben
- Captures ausführen
- optional USB- / Reconnect-Überwachung

Wichtig

Der API-Server spricht ****nicht direkt**** mit der Kamera. Die Kommunikation läuft über den Worker.

2. Start und Betriebsarten

Der Worker kann:

- mit UI laufen
- headless laufen
- über Tray verwaltet werden
- als Single Instance arbeiten
- zusätzlich einen Debug- / Capture-CLI-Modus nutzen

2.1 Sichtbare Debug- / Capture-Kommandos

Im aktuellen Stand sind in `Program.cs` direkt erkennbar:

- `capture-default`
- `capture-user`
- `--debug-capture-default`
- `--debug-capture-user`

Diese Modi dienen dazu, ohne API-Server direkt einen Capture auszulösen.

2.2 Wichtige CLI-Parameter im Debug-Capture-Modus

Erkennbar im aktuellen Stand:

- `--file / -f`
- `--path`
- `--overwrite`
- `--reset-after`
- `--iso`
- `--shutter`
- `--wb / --whitebalance / --white-balance`
- `--aperture`
- `--exposure`
- `--camera / --select`
- `--serial`
- `--skip-refresh / --no-refresh`

2.3 Capture-Modi im Debug-CLI

- `capture-default`
 - Capture mit bestehenden oder effektiven Kamera-Settings
- `capture-user`
 - Capture mit benutzerdefinierten Capture-Feldern (`applySettings = true`)

Zusätzlich ist `resetAfterShoot` vorgesehen, um Settings nach dem Shot wieder zurückzusetzen.

3. Startverhalten

Beim normalen Start sind im sichtbaren Code unter anderem diese Schritte erkennbar:

1. Restart-Args verarbeiten
2. Logging initialisieren
3. Startup-Optionen parsen
4. optional Single-Instance absichern
5. `AppSettings` und `camera-map.json` laden
6. `CameraHost` erstellen
7. `UsbReconnectWatchdog` starten
8. IPC-Server starten
9. UI- / Tray-Kontext starten

Der Pipe-Name wird aus der Konfiguration gelesen. Falls nichts gesetzt ist, wird der Standard aus `Shared.PipeNames.CommandPipe` genutzt.

4. Named Pipe IPC

4.1 Rolle des Workers

Der Worker ist der eigentliche Ausführer der IPC-Commands. Der API-Server ist die HTTP-Schicht davor.

4.2 Verfügbare Commands

Aus `Photobox.Bridge.Shared.Commands`:

- `status.get`
- `cameras.list`
- `camera.select`
- `camera.refresh`
- `liveview.start`
- `liveview.stop`
- `liveview.fps.get`
- `liveview.fps.set`
- `settings.get`
- `settings.set`
- `capture`
- `watchdog.get`
- `watchdog.set`
- `frame.wait_next`

Wichtig

- Auf Worker- / IPC-Ebene gibt es weiterhin **nur** den Capture-Command 'capture'. - Der HTTP-Endpunkt `/api/capture-liveview` ist **kein eigener Worker-Command**. - Diese Erweiterung liegt auf Ebene des API-Servers.

5. Status, Discovery und Kameraauswahl

5.1 Worker-Status

`status.get` liefert `WorkerStatusDto` mit unter anderem:

- `liveViewRunning`
- `selected`

- manufacturer
- model
- serial
- framesTotal
- frameAgeMs
- lastFrameUtc
- source
- watchdogEnabled

5.2 Kameraliste

`cameras.list` liefert `CameraInfoDto[]`:

- id
- displayName
- manufacturer
- model
- serial
- port
- isConnected

5.3 Auswahl

`camera.select` unterstützt:

- Auswahl per id
- Auswahl per serial

5.4 Refresh

`camera.refresh` verwendet `RefreshRequestDto`:

```
{ "timeoutMs": 4000 }
```

6. LiveView

Der Worker steuert LiveView per:

- `liveview.start`
- `liveview.stop`
- `liveview.fps.get`
- `liveview.fps.set`
- `frame.wait_next`

6.1 FPS

`liveview.fps.get` und `liveview.fps.set` nutzen `LiveViewFpsDto`:

```
{ "fps": 10 }
```

6.2 Frames

`frame.wait_next` arbeitet mit:

Request:

```
{ "lastSeq": 0, "timeoutMs": 1500 }
```

Response:

```
{ "seq": 12, "jpegBase64": "...", "lastFrameUtc": "..." }
```

Damit kann der API-Server den Stream pull-basiert aufbauen.

7. Settings

7.1 Lesen

`settings.get` liefert `CameraSettingsDto`:

- `iso`
- `shutter`
- `whiteBalance`
- `isoOptions`
- `shutterOptions`
- `whiteBalanceOptions`

7.2 Schreiben

`settings.set` arbeitet mit einem Partial-Patch für:

- `iso`
- `shutter`
- `whiteBalance`

8. Capture

8.1 Gemeinsames Request-Modell

Der Worker verarbeitet `CaptureRequestDto` mit diesen Feldern:

- `mode` ("file" oder "jpeg")
- `overwrite`
- `fileName`
- `path`
- `applySettings`
- `resetAfterShoot`
- `iso`
- `shutter`
- `whiteBalance`
- `aperture`
- `exposure`

8.2 Capture-Ergebnis

Bei Datei-Capture:

```
{ "ok": true, "file": "C:\\Pfad\\bild.jpg" }
```

Bei JPEG-Capture:

```
{ "contentType": "image/jpeg", "jpegBase64": "..." }
```

8.3 Bedeutung der Capture-Felder

Im aktuellen Stand sind diese drei Varianten vorgesehen:

1. **Capture mit aktuellen Kamera-Settings**
 - `applySettings = false`
2. **Capture mit benutzerdefinierten Settings**
 - `applySettings = true`
 - zum Beispiel `iso`, `shutter`, `whiteBalance`, `aperture`, `exposure`
3. **Capture mit benutzerdefinierten Settings und anschließendem Restore**
 - zusätzlich `resetAfterShoot = true`

Diese Logik ist auch im sichtbaren Debug-CLI-Modus erkennbar:

- `capture-default`
- `capture-user`

8.4 Wichtig zur LiveView-Funktion

Wichtig

Der Worker selbst hat **keinen separaten** IPC-Command wie 'capture-liveview'. Wenn nach dem Capture automatisch wieder LiveView gestartet werden soll, geschieht das im Gesamtsystem durch den **API-Server**, der nach erfolgreichem 'capture' zusätzlich 'liveview.start' aufruft.

9. Watchdog

Der Worker unterstützt:

- `watchdog.get`
- `watchdog.set`

Dafür wird `WatchdogDto` verwendet:

```
{ "enabled": true }
```

Im sichtbaren Startcode wird der `UsbReconnectWatchdog` erstellt und standardmäßig gestartet. Außerdem wird das gewünschte LiveView-Verhalten an den Watchdog übergeben.

10. Debug-CLI-Beispiele

10.1 Capture mit bestehenden Settings

```
worker.exe capture-default --camera 0 --file test.jpg
```

10.2 Capture mit benutzerdefinierten Settings

```
worker.exe capture-user --camera 0 --iso 100 --shutter 1/125 --wb Auto --file test.jpg
```

10.3 Capture mit Restore danach

```
worker.exe capture-user --camera 0 --iso 100 --shutter 1/125 --wb Auto --reset-after true --file  
↪ test.jpg
```

11. Zusammenspiel mit dem API-Server

Typischer Ablauf:

1. API-Server ruft `camera.refresh`
2. API-Server ruft `cameras.list`
3. API-Server ruft `camera.select`
4. API-Server ruft `liveview.start`
5. API-Server pollt Frames per `frame.wait_next`
6. API-Server ruft `capture`
7. optional ruft der API-Server danach zusätzlich `liveview.start`

12. Praxis-Hinweise

Checkliste

- Der Worker bleibt die zentrale Kameraschnittstelle. - Die HTTP-Erweiterung 'capture + danach liveview' liegt aktuell oberhalb des Workers im API-Server. - Für Kamera-Settings sind 'settings.get' und 'settings.set' offiziell im Shared-Protokoll vorhanden. - Für Captures sind neben 'iso', 'shutter' und 'whiteBalance' auch 'aperture' und 'exposure' im Request-Modell vorgesehen.